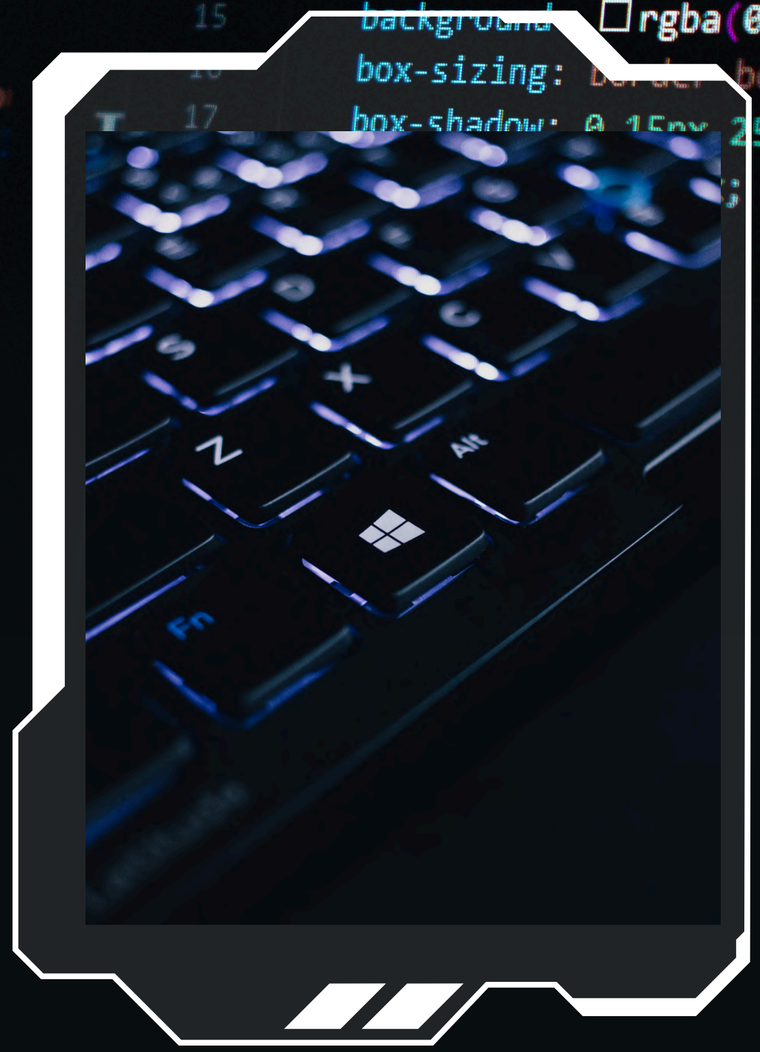
Universidad Continental

NRC: 30244

ESTRUCTURA DE DATOS

Sistema de Gestión de Procesos en C++



Integrantes:

- AGUILAR, YERSON JAIME
- AYALA SULCA, WILNER
- PARQUE NIQUEN, ANGEL LUIS
- ROMERO MARAVI, ANGHELY GABRIELA

2026

GRUPO F

GRUPO F

PLANTEAMIENTO DEL PROBLEMA Y OBJETIVOS

El Problema:



- Competencia de Recursos: Las aplicaciones compiten simultáneamente por el uso de la CPU y la memoria RAM.
- Riesgo: La falta de un control eficiente causa cuellos de botella, lentitud y fugas de memoria.

Objetivos::

- **Objetivo General: Construir un simulador en C++ que administre procesos, ordene prioridades en CPU y controle la memoria RAM.**
- **Estructuras Propias: Desarrollar la solución desde cero mediante estructuras dinámicas personalizadas, sin librerías automáticas.**



GRUPO F



Arquitectura Modular:

- **Gestor de Procesos:** Administra el registro general mediante una Lista Enlazada Simple.
- **Planificador de CPU:** Asigna los turnos de atención usando una Cola de Prioridad.
- **Gestor de Memoria RAM:** Modela la asignación y liberación física con una Pila Dinámica.

ARQUITECTURA DEL SISTEMA Y ENTIDADES (STRUCT)



Estructura Base (struct Proceso):

Atributos Clave:

- **id:** Identificador único del proceso.
- **nombre:** Descripción de la tarea.
- **prioridad:** Nivel de urgencia de atención.
- **memoriaAsignada:** Espacio reservado en KB.
- **estado:** Listo, En espera, En ejecución o Finalizado.
- **siguiente:** Puntero al próximo nodo.

GRUPO F



LISTA ENLAZADA SIMPLE: GESTOR DE PROCESOS

Funcionamiento y Operaciones:

- Estructura Base: Nodos enlazados secuencialmente mediante el puntero cabeza y enlaces siguiente.
- Funcionalidades Clave:
- Alta y Búsqueda: Inserción de tareas y consulta por ID o Nombre.
- Mantenimiento: Actualización de atributos y eliminación segura de nodos reconectando punteros.



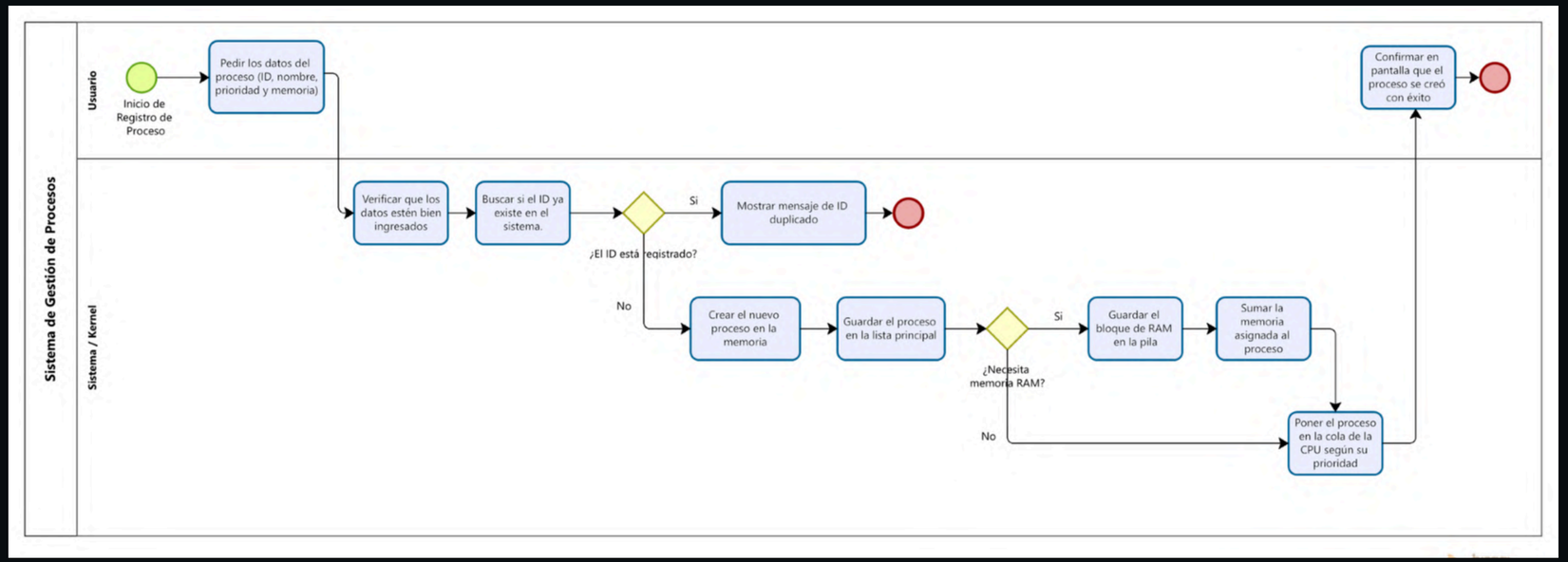
Justificación Técnica:

- Asignación Dinámica: Permite registrar procesos de forma flexible en tiempo de ejecución sin reservar un tamaño fijo predeterminado.
- Uso Eficiente de Memoria: Cada nodo consume espacio únicamente mientras el proceso permanece activo en el sistema.

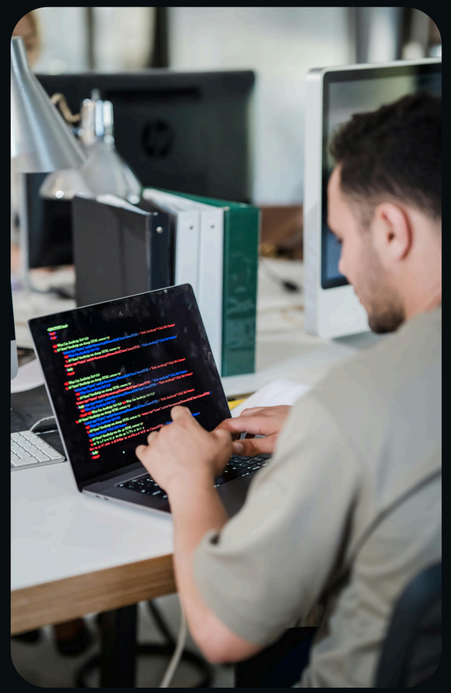
GRUPO F



LISTA ENLAZADA SIMPLE: GESTOR DE PROCESOS



GRUPO F



Criterios de Organización:

- Niveles de Urgencia: Asigna turnos según el nivel de prioridad (Nivel 1: Alta, Nivel 2: Media, Nivel 3: Baja).
- Criterio de Desempate: Si dos tareas comparten la misma prioridad, se respeta estrictamente el orden de llegada (FIFO).

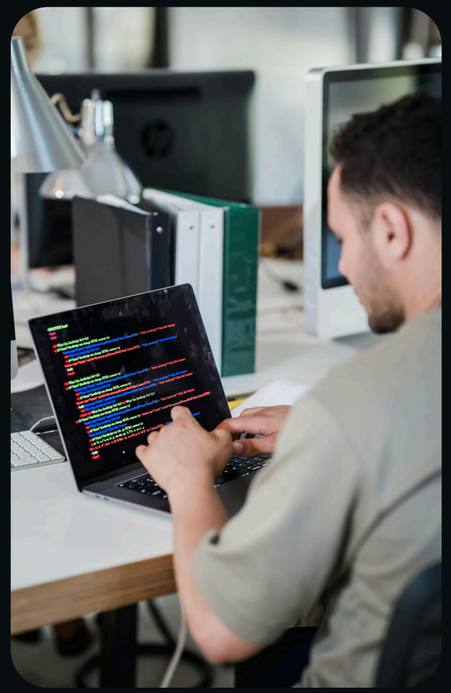


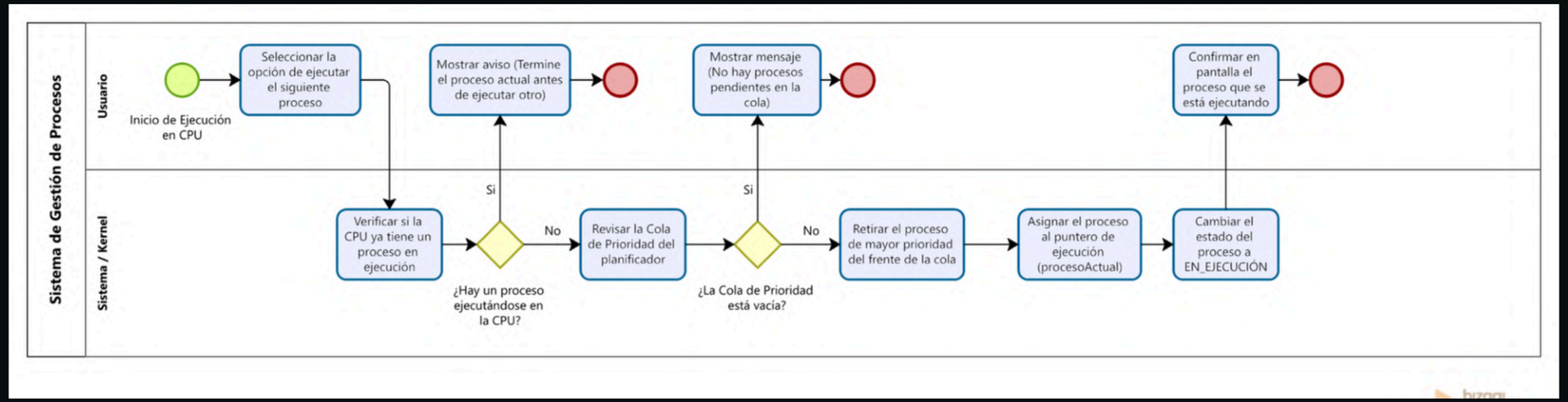
Operaciones y Despacho:

- Encolar: Inserta el nuevo nodo recorriendo la cola hasta encontrar su posición correcta por prioridad ($O(n)$).
- Desencolar / Despachar: Retira de forma inmediata ($O(1)$) el proceso del frente para entregarlo al procesador.

COLA DE PRIORIDAD: PLANIFICADOR DE CPU

GRUPO F

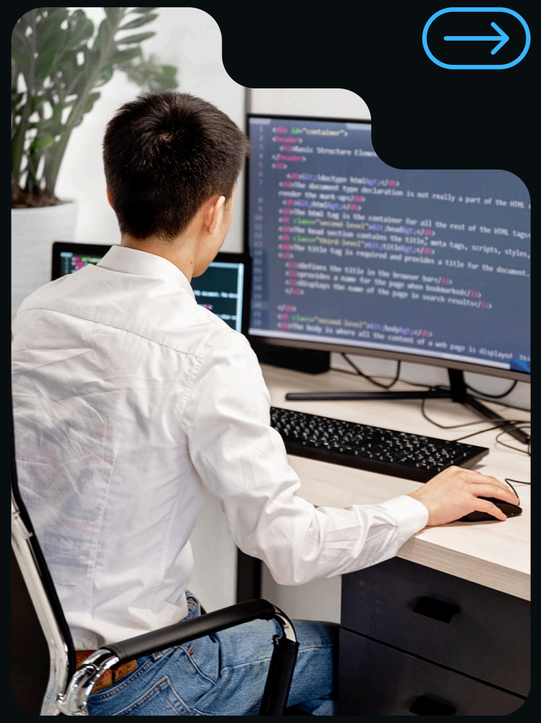




COLA DE PRIORIDAD: PLANIFICADOR DE CPU

GRUPO F

PILA DINÁMICA: GESTOR DE MEMORIA RAM





Asignación y Principio LIFO:

- Asignación por Bloques: Cada vez que un proceso reserva memoria, se crea un bloque que se ubica en el tope de la pila.
- Principio LIFO: Funciona bajo la lógica de "último en entrar, primero en salir".

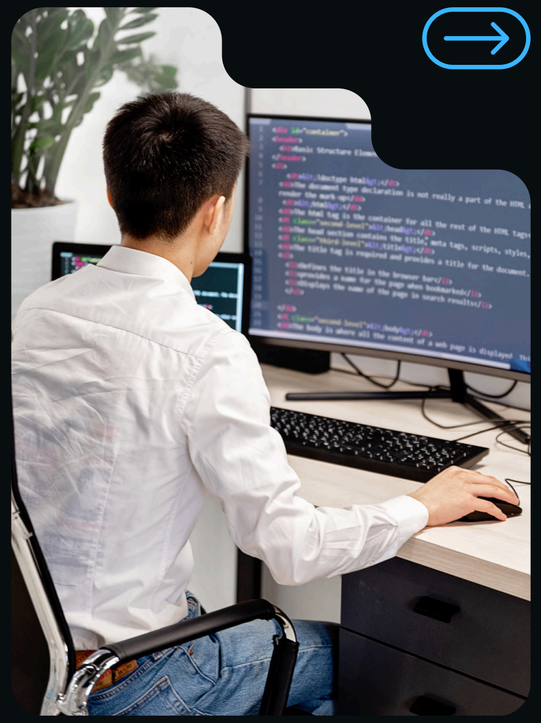
Estrategia con Pila Auxiliar:

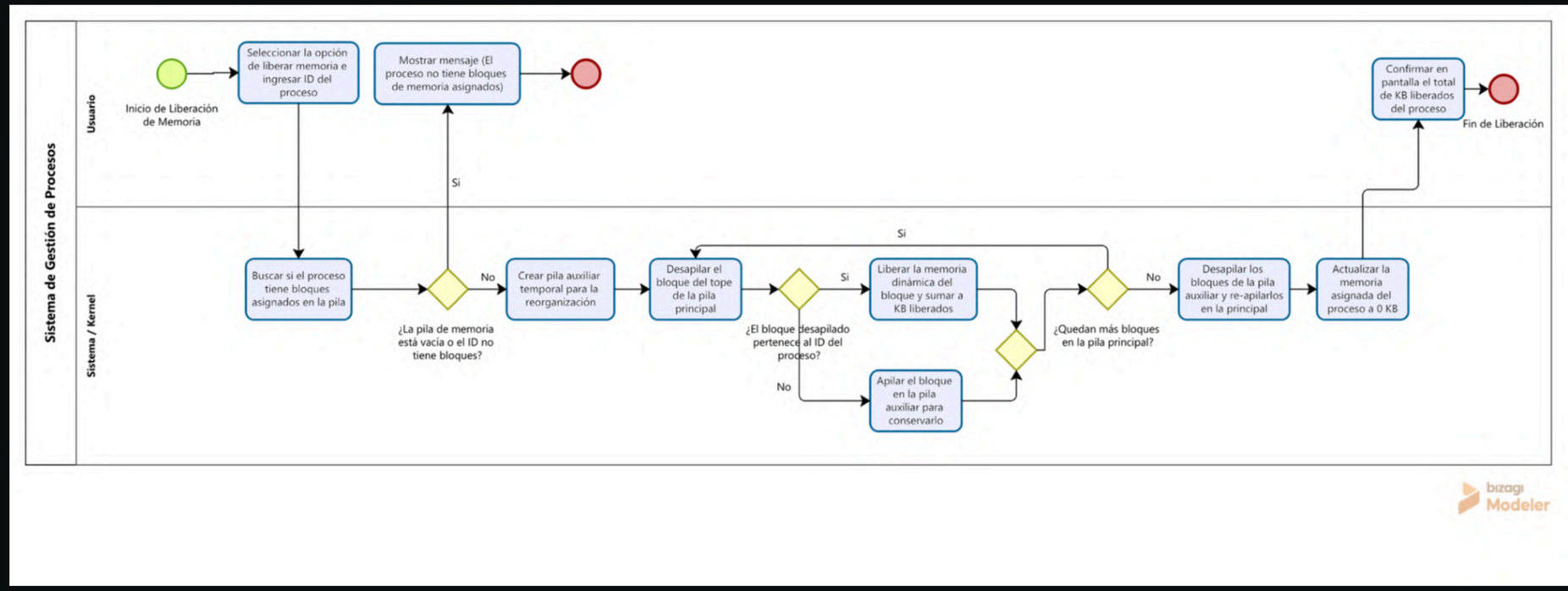
- Liberación Selectiva: Para eliminar la memoria de un proceso sin perder la de los demás, se desapilan los bloques uno a uno.
- Uso de Pila Auxiliar: Los bloques pertenecientes a otros procesos se almacenan temporalmente en una pila auxiliar.
- Restauración: Una vez liberados los bloques deseados, los elementos de la pila auxiliar se re-apilan en la principal conservando el orden original.



GRUPO F

PILA DINÁMICA: GESTOR DE MEMORIA RAM





GRUPO F



Persistencia en Archivos (procesos.txt):

- Serialización en Texto: Guarda el estado del sistema separando campos con barra vertical: 101 | Navegador | 1 | 512 | LISTO.
- Recuperación Automática: Lee la estructura del archivo y vuelve a construir los nodos en la lista y los bloques en la RAM.

PERSISTENCIA DE DATOS Y CONTROL DE ERRORES

Validación y Seguridad del Sistema:

- Control de Memoria Insuficiente: Verifica el espacio libre en RAM antes de autorizar la creación de un nuevo proceso.
- Protección del Búfer: Implementa limpieza de entrada (cin.clear()) para prevenir ciclos infinitos ante lecturas inválidas.
- Manejo de Archivos: Valida con is_open() y prohíbe el uso del carácter | en los nombres para evitar corrupción de datos.



GRUPO F

ANÁLISIS DE COMPLEJIDAD ALGORÍTMICA



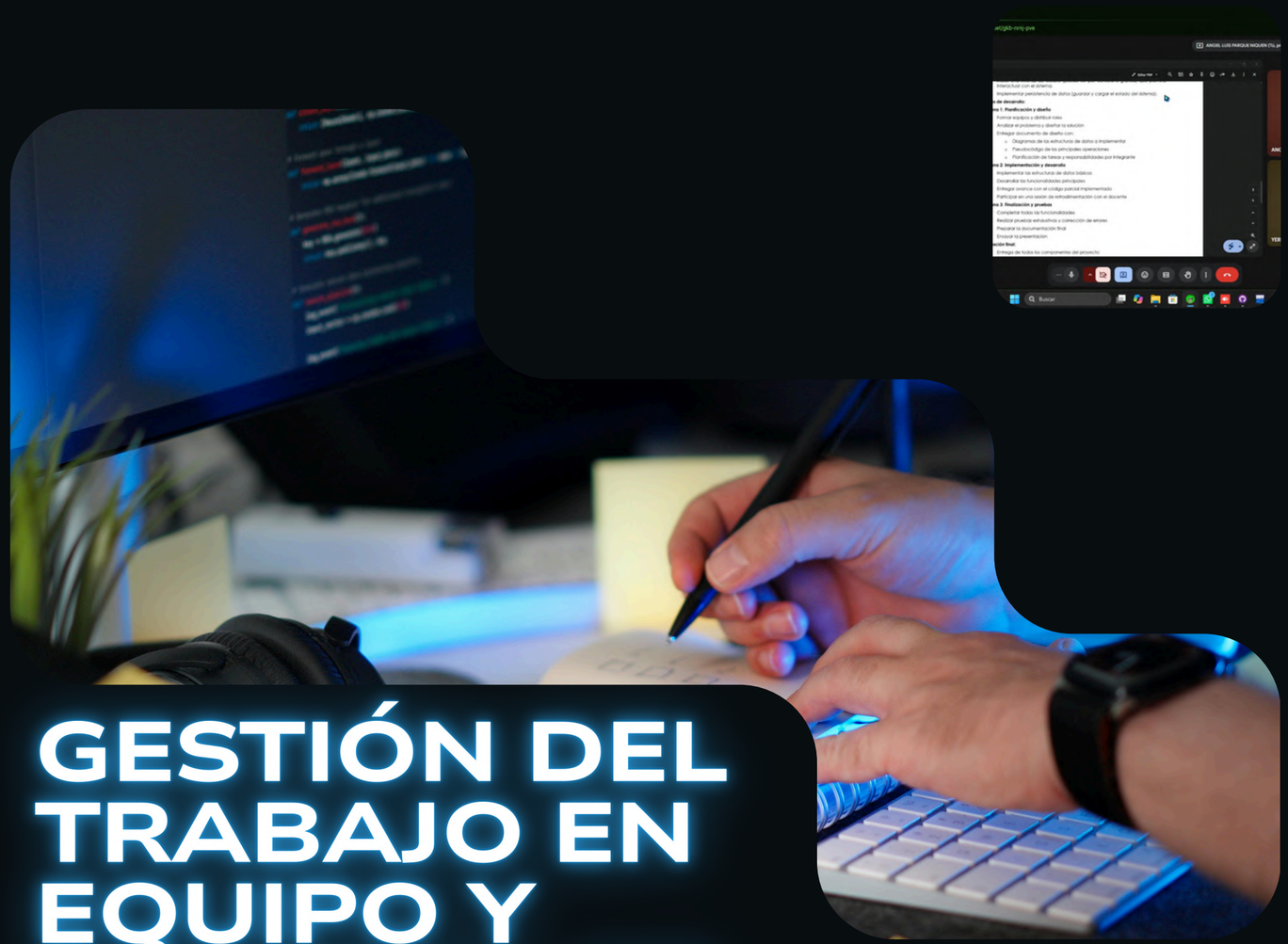
Operación	Complejidad temporal esperada	Complejidad espacial	Justificación
Búsqueda en lista enlazada	O(n)	O(1)	Puede recorrer hasta los n nodos; solo usa punteros auxiliares.
Inserción en lista enlazada al final	O(n)	O(1)	Recorre la lista hasta el último nodo y crea un nodo. Si se insertara al inicio, sería O(1).
Encolado ordenado por prioridad	O(n)	O(1)	Recorre la cola hasta encontrar la posición correcta y crea un nodo.
Desencolado	O(1)	O(1)	Retira el nodo del frente y actualiza el puntero de inicio.
Apilar / desapilar	O(1)	O(1)	Actualiza el tope y el enlace del nodo.
Liberar memoria con pila auxiliar	O(n)	O(n) temporal	Puede revisar todos los bloques; en el peor caso, la pila auxiliar conserva temporalmente n bloques.

Resumen Técnico

- Eficiencia en Ejecución: Las operaciones críticas del procesador y la pila responden en tiempo constante O(1).
- Uso de Memoria: La memoria consumida por las estructuras principales es mínima O(1), salvo el espacio temporal O(n) requerido durante la reorganización con la pila auxiliar.

GRUPO F

GESTIÓN DEL TRABAJO EN EQUIPO Y CONTROL DE VERSIONES



Repositorio y Coordinación

- Repositorio Git: Proyecto sincronizado en GitHub bajo el repositorio: https://github.com/AngelParque/Estructura-de-datos---Grupo-F
- Control de Cambios: Registros periódicos de avances para el módulo de C++, diagramas de arquitectura y redacción del informe técnico.
- Seguimiento Continuo: Coordinación de entregas e integración mediante reuniones virtuales y actas de seguimiento.

Matriz de Roles y Responsabilidades

- Angel Parque (Programador Principal C++): Desarrollo de estructuras dinámicas, algoritmo del menú interactivo y corrección de punteros.
- Anghely Romero (Líder de Documentación y Diseño): Redacción del informe, creación de diagramas Bizagi y verificación de requerimientos.
- Wilner Ayala (Administrador de Git y Desarrollador): Configuración del repositorio en GitHub, historial de avances y lógica del gestor de memoria.
- Yerson Aguilar (Especialista en Pruebas y Persistencia): Funciones de archivos de texto, validación de errores en consola y redacción de actas de reunión.

GRUPO F



CONCLUSIONES Y RECOMENDACIONES

Conclusiones del Proyecto

- Simulación Exitosa: Logramos construir un simulador completo en C++ que administra tareas, prioridades en CPU y memoria RAM usando estructuras dinámicas propias sin librerías externas.
- Estructuras Adecuadas: La lista enlazada otorga flexibilidad de registro, la cola de prioridad garantiza atención justa por urgencia y la pila auxiliar protege los bloques de RAM.
- Persistencia Asegurada: El guardado en procesos.txt garantiza que la información se conserve al cerrar y reabrir el sistema.

Recomendaciones

- Interfaz Gráfica: Incorporar una interfaz visual que muestre en bloques de colores el estado de la memoria y de la cola en lugar de texto en consola.
- Algoritmos Avanzados: Implementar tiempos de caducidad (aging) en la cola para evitar que los procesos de baja prioridad se queden en espera indefinida.
- Exportación de Reportes: Permitir la exportación de estadísticas de rendimiento hacia hojas de cálculo para generar gráficos de análisis.

Universidad Continental

Muchas gracias :)

Grupo F

